

Analysis of Algorithms (BDSF25M/A)

Terminal Assignment

Due Date: 06-May-2026

Roll No: BDSF24M029 Name: Abdullah Ayub

Grading	CLO-1	CLO-2	CLO-3	CLO-4
	/	/	/	/

In your own words, write what you have learned about the various categories of algorithms like BF, DC, DP, GR, AP, etc and problems as P, NP, NPC.

What I learned about algorithm categories:

Brute Force: BF is the simplest approach. It tries all possible solutions and then selects the best one. easy to understand and implement but usually inefficient for large inputs because running time is high.

- I learned BF is useful before designing optimized Algorithms.

example: Linear Search, maximum subarray ($O(n^2)$)

Divide & Conquer (DC): Divide & Conquer divides a problem into smaller subproblems, solves them recursively, and combines the result.

Divide $\rightarrow$ Conquer $\rightarrow$ Combine

example: merge sort, Binary Search, Quick sort

- I learned this technique often reduces complexity significantly

Dynamic Programming (DP): DP solves overlapping subproblems and stores their solutions to avoid repeated computations

2 Styles $\leftarrow$ Top Down (Memorization)
Bottom up (Tabulation)

example: Fibonacci, matrix chain multiplication, knapsack problem

- I learned that DP is very powerful for optimization problems and can convert exponential solutions into polynomial time sol.

Greedy Algorithms : greedy algo make locally best choice at each step hoping to achieve a globally optimal solution.
examples : Huffman coding , Dijkstra algo

- I learned that greedy Algos are usually faster and simpler but they work only when the greedy choice property exist.

Approximation Algos : ~~AP~~ AP are used for very difficult problems where finding the exact optimal solution is impractical provides : near optimal solution , within acceptable time.

example : Travelling sales problem (TSP)

I learned that AP are important for NP-hard problems where exact solutions are too expensive.

What I Learned about Problem classes :

P Problem : P stands for polynomial time

These are problems that can be solved efficiently using algorithms with polynomial running time.

examples : Sorting , Searching , BFS , DFS

I learned that P problems are considered computationally feasible.

NP Problems : NP stands for Non deterministic Polynomial time

These problems may not be easy to solve quickly, but if a solution is given it can be verified quickly

example : Subset Sum , TSP (decision)

- I learned that every P problem is also in NP

$$P \subseteq NP$$

NPC problem : NP-complete.

NPC problems are the hardest problems inside NP.

→ They belong to NP, every NP problem can be reduced to them

example : SAT , Hamiltonian cycle

I learned that if one NPC problem gets a polynomial time then all NP problems can also be solved in polynomial time.

NP Hard : NP hard are at least as hard as NPC (Optimization of TSP)